

Введение в глубокое обучение

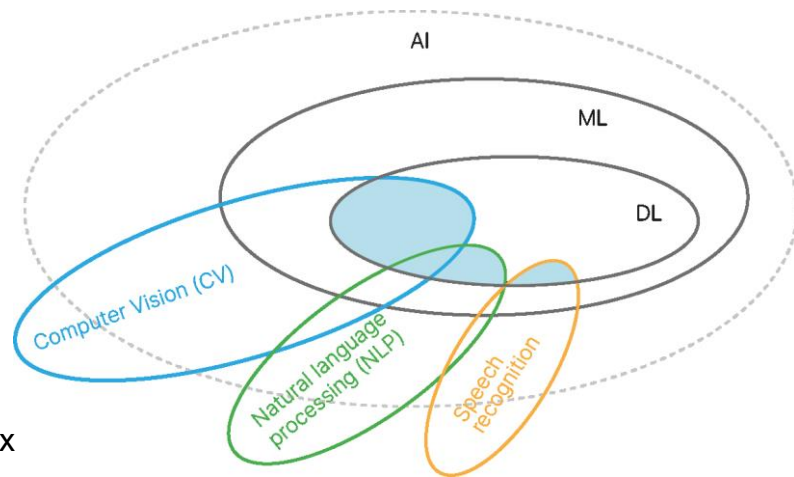
Введение в глубокое обучение

Введение в глубокое обучение

Искусственный интеллект (AI/ИИ) — область IT/Computer science, связанная с моделированием интеллектуальных или творческих видов человеческой деятельности.

Машинное обучение (ML) — подраздел ИИ, связанный с разработкой алгоритмов и статистических моделей, которые компьютерные системы используют для выполнения задач без явных инструкций.

Глубокое обучение (Deep Learning, DL) — совокупность методов машинного обучения, основанных на искусственных нейронных сетях и обучении представлением (feature/representation learning). Данный класс методов автоматически выделяет из необработанных данных необходимые признаки (представления), в отличие от методов ML, в которых признаки создают люди вручную (feature engineering).



Глубокое обучение

История



Задачи в глубоком обучении

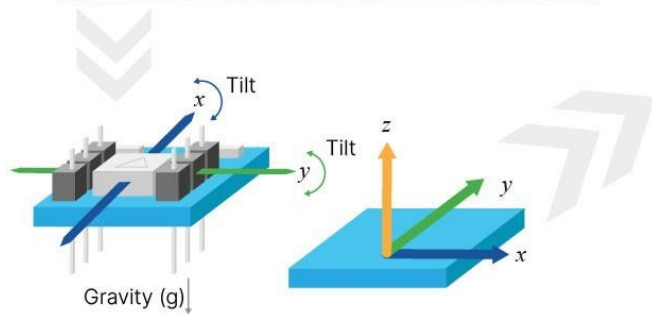
Классификация изображений

Задача классификации изображений состоит в том, чтобы поставить в соответствие изображению класс объектов

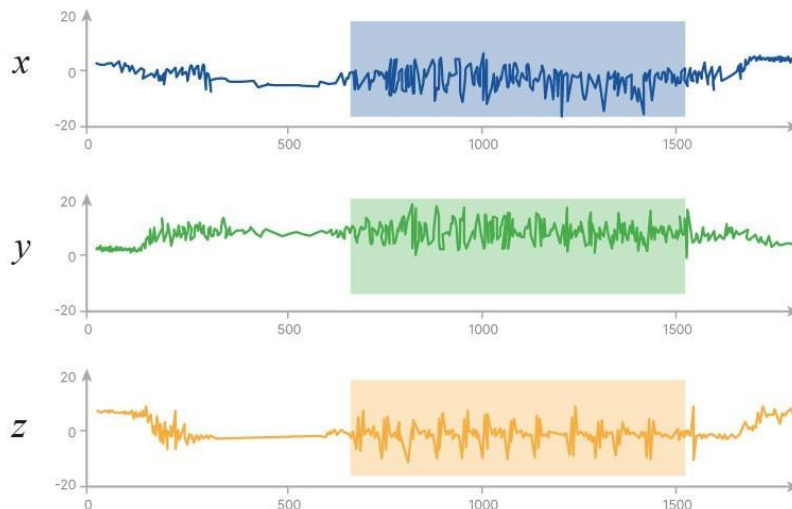


Задачи в глубоком обучении

Классификация или регрессия временных рядов



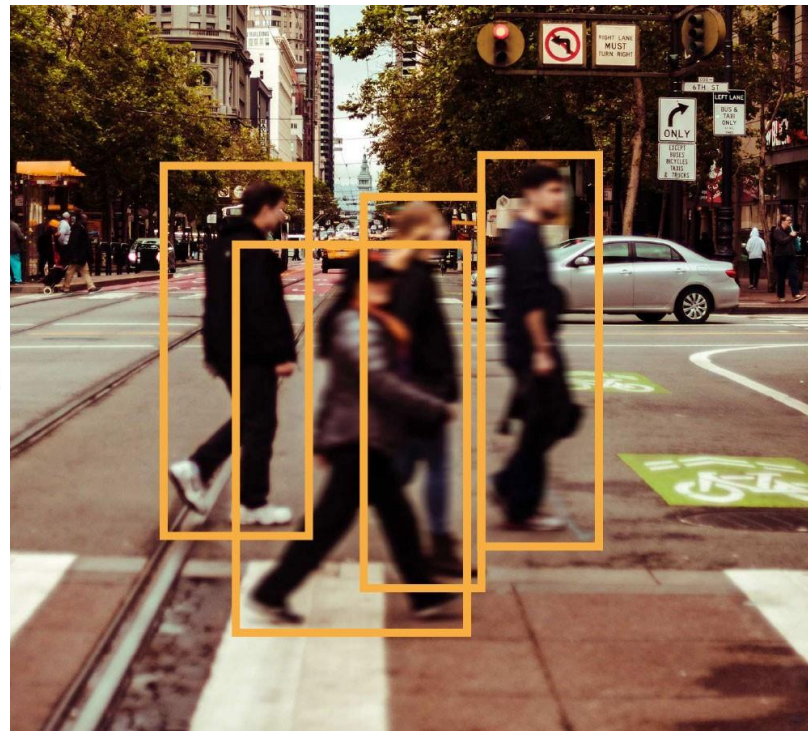
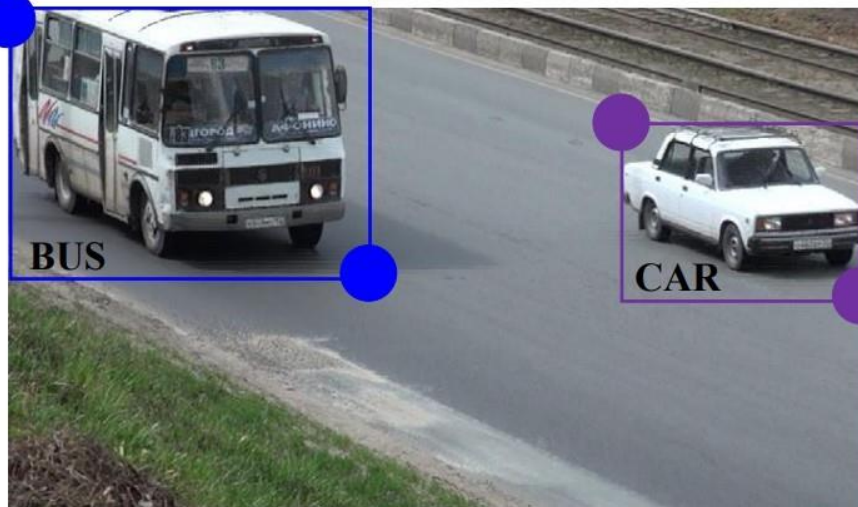
Accelerometer Sensor Data



Задачи в глубоком обучении

Детектирование

Задача детектирования объектов состоит в том, чтобы определить положение прямоугольника, охватывающего объект заданного класса



Задачи в глубоком обучении

Сегментация

Задача семантической сегментации состоит в том, чтобы каждому пикселю изображения поставить в соответствие класс объектов, которому он принадлежит



Оригинал

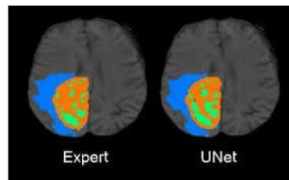


Разметка

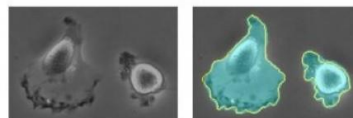


Результат сегментации

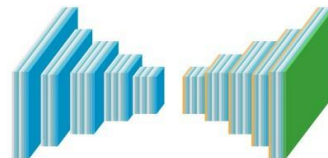
Поиск аномалий



Трекинг клеток



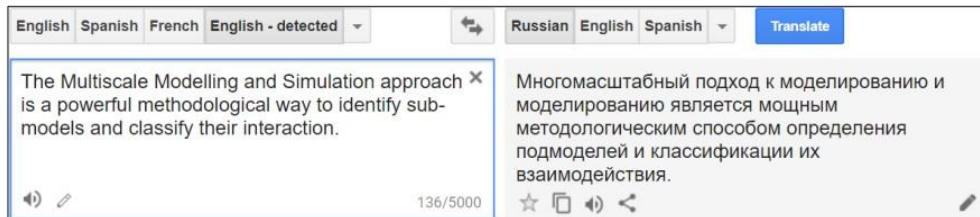
Сегментация



Задачи в глубоком обучении

Генерация текста и изображений

□ Google Translate



Задачи в глубоком обучении

Перенос стиля



Примеры переноса стиля

Какие есть данные?

Существуют различные типы данных.

1. Последовательности (важен порядок данных, время):

- **временные ряды** (речь, мозговая активность, котировки);
- **текст**.

2. Пространственно-структурированная информация (преобразуется к векторам чисел):

- **изображения** (пиксели);
- **видео** (пиксели + время);
- **3D** (воксели).

3. Статистика:

- **табличные данные** (признаки).

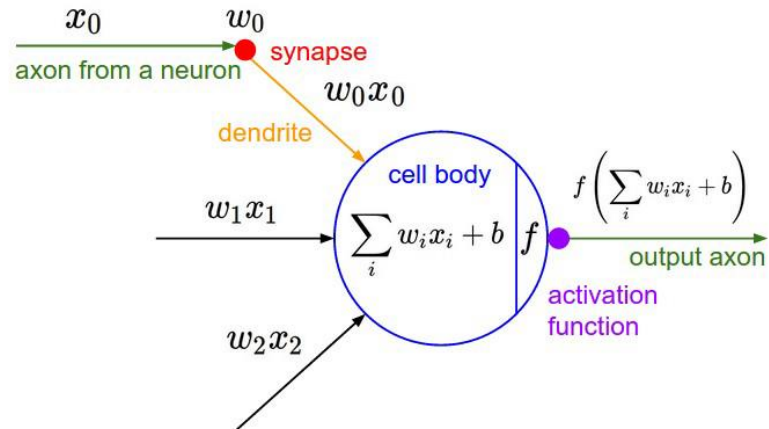
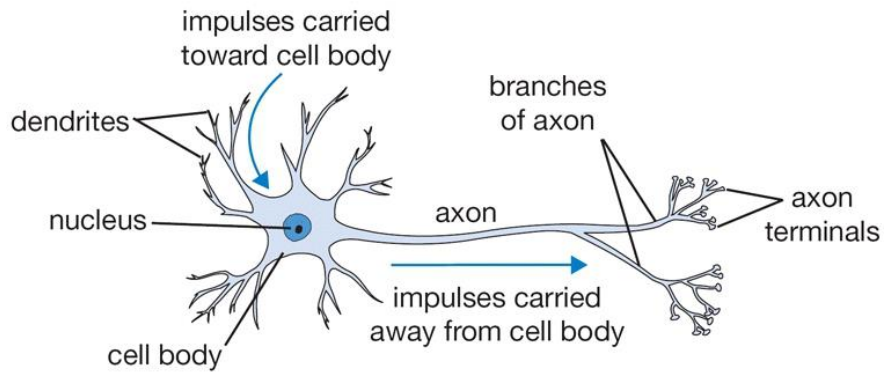
Для работы с различными типами данных используют разные типы моделей:

Табличный — классические ML модели либо полносвязные NN, трансформеры.

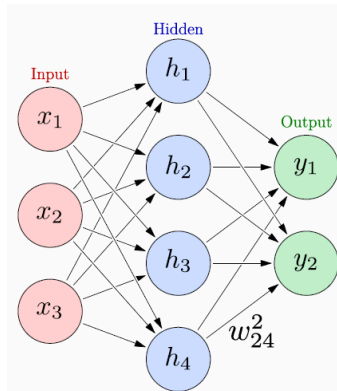
Последовательности — рекуррентные сети + свёртка, трансформеры.

Изображения/видео — 2,3 .. ND свёрточные сети, трансформеры.

Нейрон

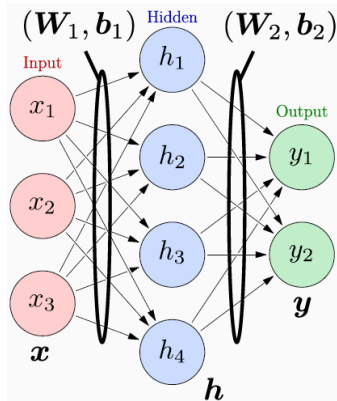


Полносвязная нейронная сеть



$$\begin{aligned}h_1 &= g_1(w_{11}^1x_1 + w_{12}^1x_2 + w_{13}^1x_3 + b_1^1) \\h_2 &= g_1(w_{21}^1x_1 + w_{22}^1x_2 + w_{23}^1x_3 + b_2^1) \\h_3 &= g_1(w_{31}^1x_1 + w_{32}^1x_2 + w_{33}^1x_3 + b_3^1) \\h_4 &= g_1(w_{41}^1x_1 + w_{42}^1x_2 + w_{43}^1x_3 + b_4^1)\end{aligned}$$

$$\begin{aligned}y_1 &= g_2(w_{11}^2h_1 + w_{12}^2h_2 + w_{13}^2h_3 + w_{14}^2h_4 + b_1^2) \\y_2 &= g_2(w_{21}^2h_1 + w_{22}^2h_2 + w_{23}^2h_3 + w_{24}^2h_4 + b_2^2)\end{aligned}$$



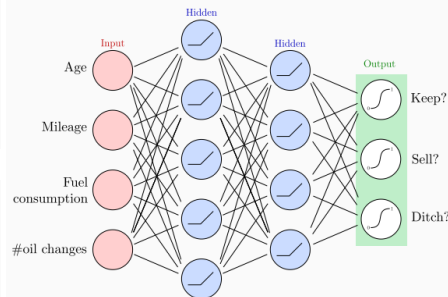
$$\begin{aligned}h_1 &= g_1(w_{11}^1x_1 + w_{12}^1x_2 + w_{13}^1x_3 + b_1^1) \\h_2 &= g_1(w_{21}^1x_1 + w_{22}^1x_2 + w_{23}^1x_3 + b_2^1) \\h_3 &= g_1(w_{31}^1x_1 + w_{32}^1x_2 + w_{33}^1x_3 + b_3^1) \\h_4 &= g_1(w_{41}^1x_1 + w_{42}^1x_2 + w_{43}^1x_3 + b_4^1)\end{aligned}$$

$$h = g_1(W_1x + b_1)$$

$$\begin{aligned}y_1 &= g_2(w_{11}^2h_1 + w_{12}^2h_2 + w_{13}^2h_3 + w_{14}^2h_4 + b_1^2) \\y_2 &= g_2(w_{21}^2h_1 + w_{22}^2h_2 + w_{23}^2h_3 + w_{24}^2h_4 + b_2^2)\end{aligned}$$

$$y = g_2(W_2h + b_2)$$

• Typical architecture:



- Hidden layer:

$$\text{ReLU}(a) = \max(a, 0)$$

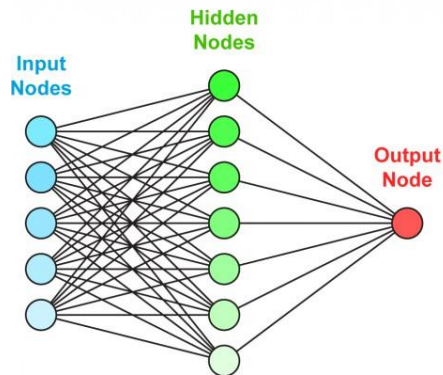
- Output layer:

$$\text{softmax}(a)_k = \frac{\exp(a_k)}{\sum_{\ell=1}^K \exp(a_\ell)}$$

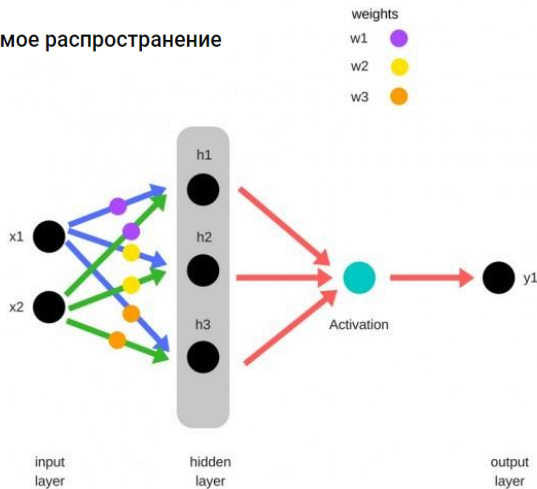
Принцип обучения

Искусственная нейронная сеть состоит из трех компонентов:

- Входной слой;
- Скрытые (вычислительные) слои;
- Выходной слой.



Прямое распространение



Зададим начальные веса случайным образом:

- $w1$
- $w2$

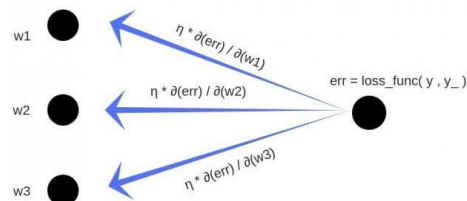
Умножим входные данные на веса для формирования скрытого слоя:

- $h1 = (x1 * w1) + (x2 * w1)$
- $h2 = (x1 * w2) + (x2 * w2)$
- $h3 = (x1 * w3) + (x2 * w3)$

Выходные данные из скрытого слоя передается через нелинейную функцию (функцию активации), для получения выхода сети:

- $y_- = \text{fn}(h1, h2, h3)$

Обратное распространение



Полученный результат затем вычитается из соответствующих весов.

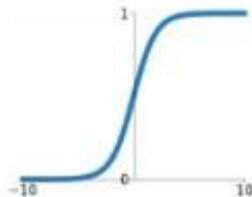
В результате получаются следующие обновленные веса:

- $w1 = w1 - (\eta * \partial(\text{err}) / \partial(w1))$
- $w2 = w2 - (\eta * \partial(\text{err}) / \partial(w2))$
- $w3 = w3 - (\eta * \partial(\text{err}) / \partial(w3))$

Функции активации

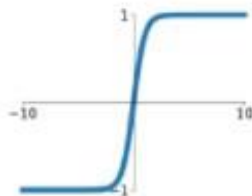
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



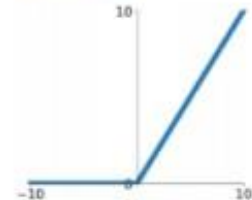
tanh

$$\tanh(x)$$



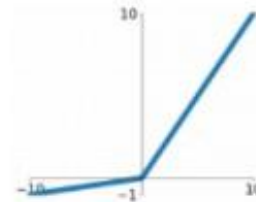
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

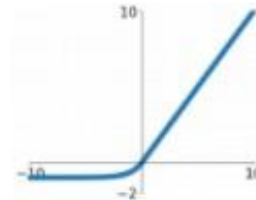


Maxout

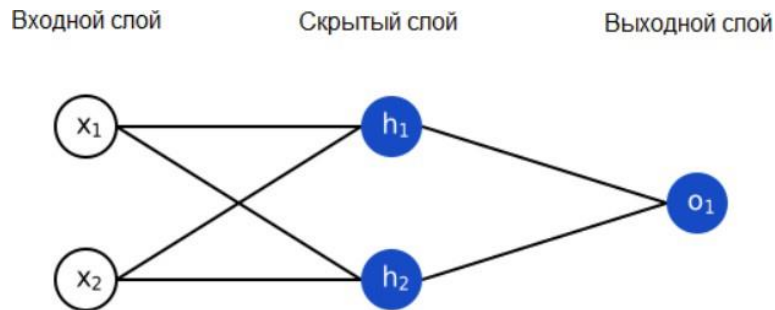
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Прямое распространение



$$x_1 \rightarrow x_1 * w_1, x_2 \rightarrow x_2 * w_2$$

$$x_1 * w_1 + x_2 * w_2 + b$$

$$y = f(x_1 * w_1 + x_2 * w_2 + b)$$

Все нейроны имеют одинаковые веса $w=[0,1]$, одинаковые пороговые значения $b=0$, и одинаковую функцию активации - сигмоиду. На вход подаем $x=[2,3]$.

$$h_1 = h_2 = f(w \cdot x + b) = f((0 * 2) + (1 * 3) + 0) = f(3) = 0.9526$$

$$o_1 = f(w \cdot [h_1, h_2] + b) = f((0 * h_1) + (1 * h_2) + 0) = f(0.9526) = \mathbf{0.7216}$$

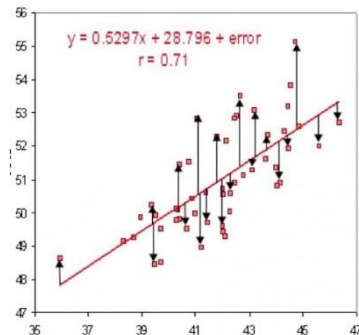
Функции потерь

Квадратичная (среднеквадратичное отклонение): $MSE = \frac{1}{n} \sum_{i=1}^n (y_{true} - y_{pred})^2$

Бинарная кросс-энтропия: $H_p(q) = -\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$

Расстояние Кульбака — Лейблера или прирост информации:

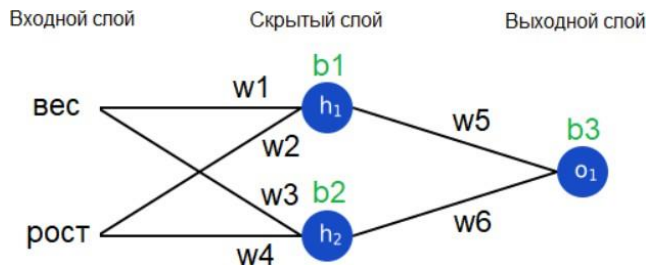
$$D_{KL}(q||p) = H_p(q) - H(q) = \sum_{c=1}^C q(y_c) \cdot [\log(q(y_c)) - \log(p(y_c))]$$



ИМЯ	ytrue	ypred	(ytrue-ypred) ²
Алиса	1	0	1
Боб	0	0	0
Чарли	0	0	0
Диана	1	0	1

$$MSE = \frac{1}{4} (1 + 0 + 0 + 1) = 0.5$$

Обратное распространение



$$L(w_1, w_2, w_3, w_4, w_5, w_6, b_1, b_2, b_3)$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_{pred}} * \frac{\partial y_{pred}}{\partial w_1}$$

$$\frac{\partial L}{\partial y_{pred}} = \frac{\partial (1 - y_{pred})^2}{\partial y_{pred}} = -2(1 - y_{pred})$$

$$y_{pred} = o_1 = f(w_5 h_1 + w_6 h_2 + b_3)$$

$$\frac{\partial y_{pred}}{\partial w_1} = \frac{\partial y_{pred}}{\partial h_1} * \frac{\partial h_1}{\partial w_1}$$

$$\frac{\partial y_{pred}}{\partial h_1} = w_5 * f'(w_5 h_1 + w_6 h_2 + b_3)$$

$$h_1 = f(w_1 x_1 + w_2 x_2 + b_1)$$

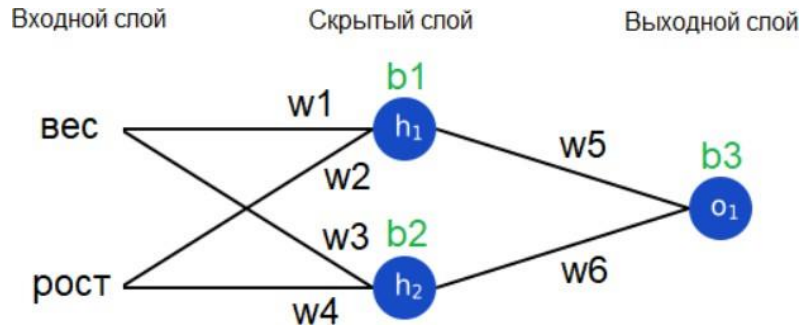
$$\frac{\partial h_1}{\partial w_1} = x_1 * f'(w_1 x_1 + w_2 x_2 + b_1)$$

$$f(x) = \frac{1}{1 + e^{-x}}$$

$$f'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = f(x) * (1 - f(x))$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_{pred}} * \frac{\partial y_{pred}}{\partial h_1} * \frac{\partial h_1}{\partial w_1}$$

Обратное распространение



$$h_1 = f(w_1x_1 + w_2x_2 + b_1) = f(-2 + (-1) + 0) = 0.0474$$

$$h_2 = f(w_3x_1 + w_4x_2 + b_2) = 0.0474$$

$$o_1 = f(w_5h_1 + w_6h_2 + b_3) = f(0.0474 + 0.0474 + 0) = 0.524$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_{pred}} * \frac{\partial y_{pred}}{\partial h_1} * \frac{\partial h_1}{\partial w_1}$$

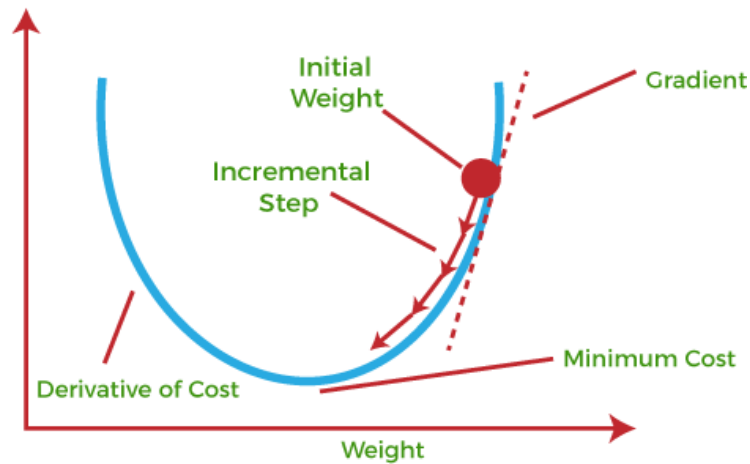
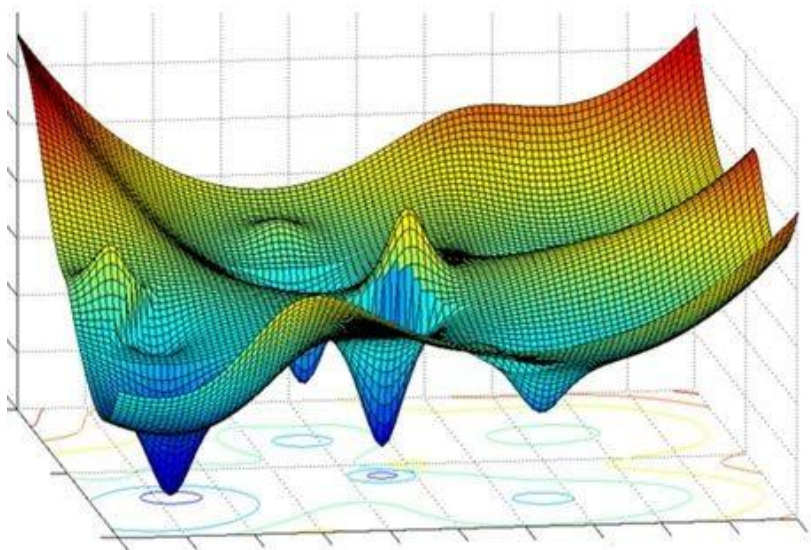
$$\frac{\partial L}{\partial y_{pred}} = -2(1 - y_{pred}) = -2(1 - 0.524) = -0.952$$

$$\frac{\partial y_{pred}}{\partial h_1} = w_5 * f'(w_5h_1 + w_6h_2 + b_3) = 1 * f'(0.0474 + 0.0474 + 0) = f(0.948)(1 - f(0.948)) = 0.249$$

$$\frac{\partial h_1}{\partial w_1} = x_1 * f'(w_1x_1 + w_2x_2 + b_1) = -2 * f'(-2 + (-1) + 0) = -2 * f(-3) * (1 - f(-3)) = -0.0904$$

$$\frac{\partial L}{\partial w_1} = -0.952 * 0.249 * -0.0904 = 0.0214$$

Градиентный спуск

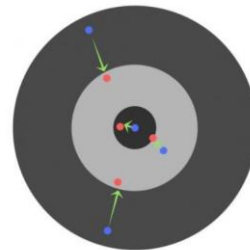
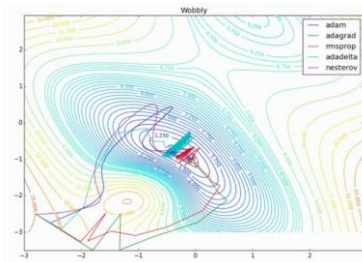


$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

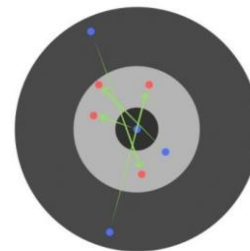
Обучение: стохастический градиентный спуск

$$w_1 \leftarrow w_1 - \eta \frac{\partial L}{\partial w_1}$$

$\eta(eta)$ — константа, называемая **скоростью обучения** (*learning rate*).



Низкий learning rate



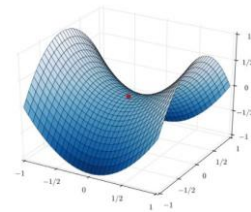
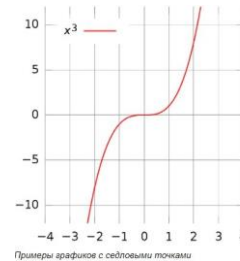
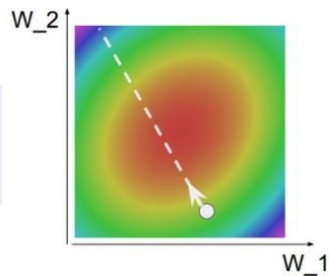
Высокий learning rate

Процесс обучения сети будет выглядеть примерно так:

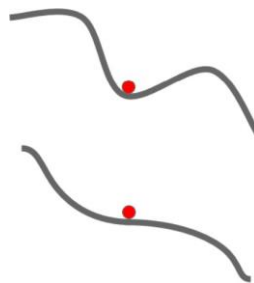
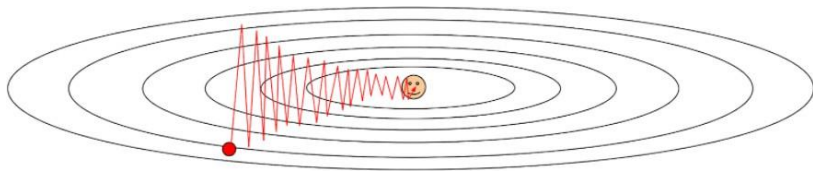
- Выбираем одно наблюдение из набора данных. Именно то, что мы работаем только с одним наблюдением, делает наш градиентный спуск стохастическим.
- Считаем все частные производные функции потерь по всем весам и порогам (dL/dw_1 , dL/dw_2 и т.д.)
- Используем формулу обновления, чтобы обновить значения каждого веса и порога.
- Снова переходим к шагу 1.

Оптимизация: подводные камни

```
# Vanilla Gradient Descent  
  
while True:  
    weights_grad = evaluate_gradient(loss_fun, data, weights)  
    weights += - step_size * weights_grad # perform parameter update
```



Примеры графиков с седловыми точками



Улучшенная оптимизация:

SGD + Momentum
Nesterov Accelerated Gradient
AdaGrad
RMSProp
Adam

Выбор скорости обучения

Существует три распространенных способа снижения learning rate:

Пошаговое затухание: снижение скорости на некоторую величину каждые несколько эпох. Типичные варианты: уменьшение в два раза каждые 5 эпох или в 0.1 раз каждые 20 эпох. Конкретные цифры сильно зависят от задачи и модели.

Экспоненциальное затухание и затухание 1/t:

Здесь α_0 и k — гиперпараметры, а t — номер итерации.

Экспоненциальное затухание: $\alpha = \alpha_0 e^{-kt}$

Затухание 1/t: $\alpha = \alpha_0 / (1 + kt)$

Обучение

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
X_tiny = X_train[:20] # take 20 examples
y_tiny = y_train[:20]
best_model, stats = trainer.train(X_tiny, y_tiny, X_tiny, y_tiny,
                                  model, two_layer_net,
                                  num_epochs=200, reg=0.0,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = False,
                                  learning_rate=1e-3, verbose=True)
```

```
Finished epoch 1 / 200: cost 2.302603, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 2 / 200: cost 2.302258, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 3 / 200: cost 2.301849, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 4 / 200: cost 2.301196, train: 0.650000, val 0.650000, lr 1.000000e-03
Finished epoch 5 / 200: cost 2.300044, train: 0.650000, val 0.650000, lr 1.000000e-03
Finished epoch 6 / 200: cost 2.297864, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 7 / 200: cost 2.293595, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 8 / 200: cost 2.285096, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 9 / 200: cost 2.268094, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 10 / 200: cost 2.234787, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 11 / 200: cost 2.173187, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 12 / 200: cost 2.076862, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 13 / 200: cost 1.974090, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 14 / 200: cost 1.895885, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 15 / 200: cost 1.820876, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 16 / 200: cost 1.737430, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 17 / 200: cost 1.642356, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 18 / 200: cost 1.535239, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 19 / 200: cost 1.421527, train: 0.600000, val 0.600000, lr 1.000000e-03
```

```
Finished epoch 195 / 200: cost 0.002694, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 196 / 200: cost 0.002674, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 197 / 200: cost 0.002655, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 198 / 200: cost 0.002635, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 199 / 200: cost 0.002617, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 200 / 200: cost 0.002597, train: 1.000000, val 1.000000, lr 1.000000e-03
finished optimization. best validation accuracy: 1.000000
```

✓ for (int i=0;i<maxEpoch;i++)
for (int j=0;j<trainSet;j++)

✗ for (int j=0;j<trainSet;j++)
for (int i=0;i<maxEpoch;i++)

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e-6, verbose=True)
```

```
Finished epoch 1 / 10: cost 2.302576, train: 0.080000, val 0.103000, lr 1.000000e-06
Finished epoch 2 / 10: cost 2.302582, train: 0.121000, val 0.124000, lr 1.000000e-06
Finished epoch 3 / 10: cost 2.302558, train: 0.119000, val 0.138000, lr 1.000000e-06
Finished epoch 4 / 10: cost 2.302519, train: 0.127000, val 0.151000, lr 1.000000e-06
Finished epoch 5 / 10: cost 2.302517, train: 0.158000, val 0.171000, lr 1.000000e-06
Finished epoch 6 / 10: cost 2.302518, train: 0.179000, val 0.172000, lr 1.000000e-06
Finished epoch 7 / 10: cost 2.302466, train: 0.180000, val 0.176000, lr 1.000000e-06
Finished epoch 8 / 10: cost 2.302452, train: 0.175000, val 0.185000, lr 1.000000e-06
Finished epoch 9 / 10: cost 2.302459, train: 0.206000, val 0.192000, lr 1.000000e-06
Finished epoch 10 / 10: cost 2.302420, train: 0.190000, val 0.192000, lr 1.000000e-06
finished optimization. best validation accuracy: 0.192000
```

Оценка обучения и оптимизация гиперпараметров

train

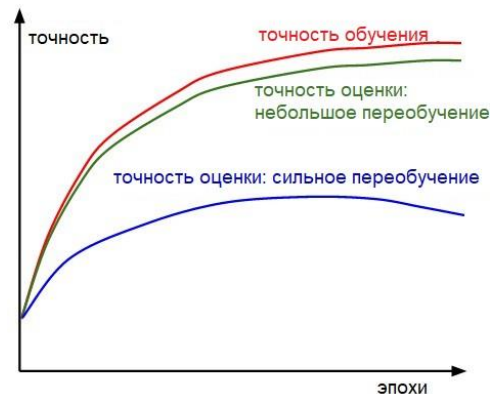
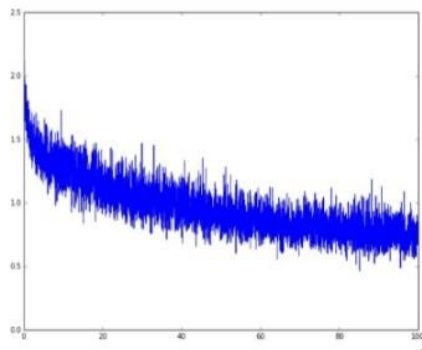
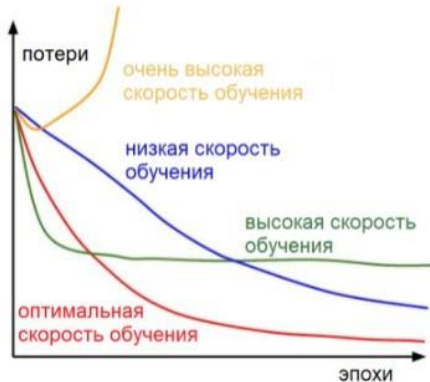
- для обучения

val

- для проверки качества во время обучения

test

- для проверки качества в конце обучения



Резюме

- Исторические предпосылки нейросетевых методов
- Типы задач в глубоком обучении
- Типы данных в глубоком обучении
- Модель нейрона
- Функции активации
- Градиентный спуск
- Принцип обучения нейронной сети
- Батч, эпоха, итерация
- Кривые обучения
- Скорость обучения

Использованные материалы

- Материалы курса «Основы глубокого обучения», автор – Мошкова А.А.
- [Машинное обучение для начинающих: Введение в нейронные сети](#)